Introduction to Programming

W04 Core Data Structures

Stephan Krusche

5 November 2025
Technical University of Munich

Schedule

#	Date	Subject
1	15.10.25	Introduction
2	22.10.25	Control Flow in Programming
3	29.10.25	**In-Depth Core Concepts***
4	**05.11.25**	**Core Data Structures**
5	12.11.25	Code Reuse and Structure
6	19.11.25	Type Flexibility and Safety
7	26.11.25	**In-Depth Object Orientation***
8	03.12.25	Functional Programming Essentials
9	10.12.25	Algorithms and Data Handling
10	17.12.25	Programming Languages
11	07.01.26	Graphical User Interfaces
12	14.01.26	Recursion
13	21.01.26	Concurrency
14	28.01.26	Beyond Programming
15	04.02.26	Course Review

*** Repetition**

Roadmap

- **Context**
  - Understand the basics of object oriented programming
  - Use basic control structures (**`if`**, **`switch`**, **`for`**, **`while`**)
- **Learning goals**
  - Implement and use simple abstract data types such as **`List`**, **`Stack`** and **`Queue`**
  - Explain how **stacks** and **queues** work internally
  - Explain the differences between a **`List`** and an **`Array`**
  - Distinguish between **`List`** and **`Array`** based data types
  - Use simple built-in collection types such as **`List`** and **`Set`**
  - Iterate using the **`for each`** loop

Abstract data types

- Specify only the operations
- Hide details
  - Internal data structure
  - Implementation of the operations

→ Information Hiding

Motivation

- Preventing illegal access to the data structure
- Decouple sub-problems for
  - Implementation
  - Debugging
  - Maintenance
- Easy exchange of implementations (rapid prototyping)

Outline

- **List (part 1)**
- List (part 2)
- Stack
- Queue

Java collection types

- Unified architecture for representation and manipulation of collections
- Reduction of programming effort
- Increase in the performance of operations on collections
- Interoperability of independent collections
- Most used types: **`List`**, **`Set`**, **`Queue`**, **`Map`**

http://download.oracle.com/javase/tutorial/collections/index.html

`java.util.List<E>`

Generic type

- The **`List`** **interface** represents an ordered sequence of objects

More about interfaces later

List

`+ add(element: E): boolean`
`+ clear(): void`
`+ contains(element: E): boolean`
`+ get(index: int): E`
`+ isEmpty(): boolean`
`+ remove(index: int): E`
`+ remove(element: E): boolean`
`+ set(index: int, element: E): E`
`+ size(): int`

Explanations

- **`add(element: E): boolean`** - Adds an element to the end of the list and returns true if it was successful.
- **`clear(): void`** - Removes all elements from the list, leaving it empty.
- **`contains(element: E): boolean`** - Checks if the list contains a specific element and returns true if it does.
- **`get(index: int): E`** - Retrieves the element at the specified position in the list.
- **`isEmpty(): boolean`** - Returns true if the list has no elements in it.
- **`remove(index: int): E`** - Removes and returns the element at the specified position in the list.
- **`remove(element: E): boolean`** - Removes the first occurrence of a specific element from the list, returning true if it was removed.
- **`set(index: int, element: E): E`** - Replaces the element at the specified position in the list with a new element, returning the original element.
- **`size(): int`** - Returns the number of elements currently in the list.

java.util.List<E>

Generic type

- Offers different implementations
- Example: ArrayList

```
import java.util.*;

class Playground {
    public static void main(String[] args) {
        List<String> words = new ArrayList<String>();
        words.add("This");
        words.add("sentence");
        words.add("has");
        words.add("five");
        words.add("words");

        System.out.println(words.get(3));
        System.out.println(words.contains("sentence"));
        System.out.println(words.indexOf("This"));

        for (String word : words) {
            System.out.println(word);
        }
    }
}
```

for each loop

Output

```
five
true
0
This
sentence
has
five
words
```

java.util.Set<E>

Generic type

- The **Set** interface represents an unordered collection of objects that contains no duplicate elements (each element is unique)

Set
+ add(element: E): boolean + clear(): void + contains(element: E): boolean + isEmpty(): boolean + remove(element: E): E + size(): int

Explanations

- **`add(element: E): boolean`** - Adds an element to the set and returns true if it was successful.
- **`clear(): void`** - Removes all elements from the set, leaving it empty.
- **`contains(element: E): boolean`** - Checks if the set contains a specific element and returns true if it does.
- **`isEmpty(): boolean`** - Returns true if the set has no elements in it.
- **`remove(element: E): boolean`** - Removes a specific element from the set if it exists, returning true if it was removed.
- **`size(): int`** - Returns the number of elements currently in the list.

java.util.Set<E>

Generic type

- Offers different implementations
- Example: HashSet

```
import java.util.*;

class Playground {
    public static void main(String[] args) {
        Set<String> words = new HashSet<String>();
        words.add("are");
        words.add("you");
        words.add("sure");
        words.add("you");
        words.add("are");
        words.add("right");

        System.out.println(words.size());

        for (String word : words) {
            System.out.println(word);
        }
    }
}
```

for each loop

Output

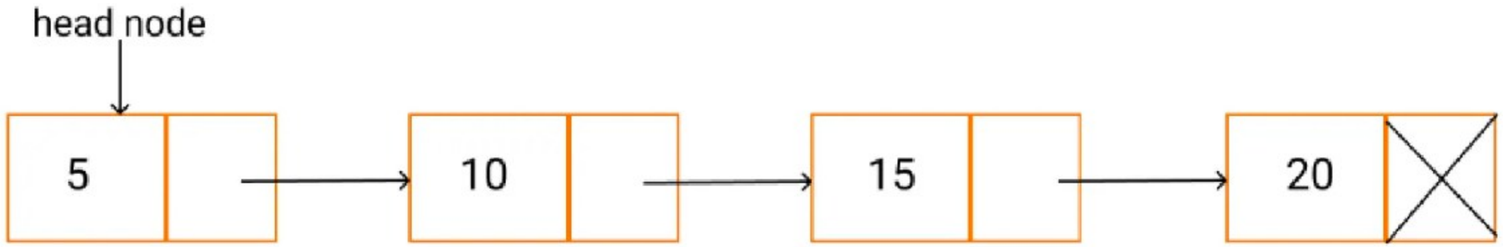
```
4
sure
are
right
you
```

Only unique elements

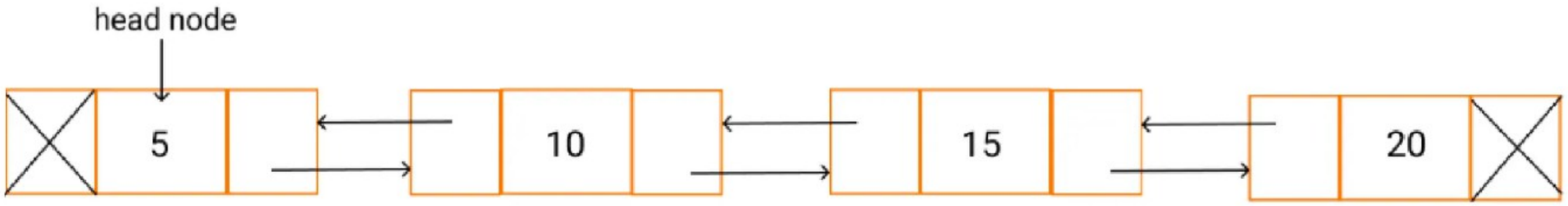
No specific order

Linked lists

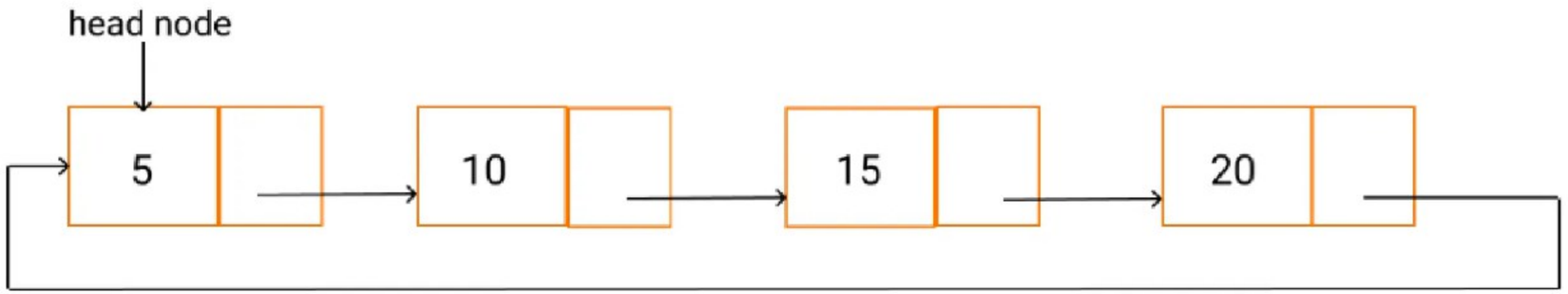
Singly Linked List



Doubly Linked List



Circular Linked List



`ArrayList` vs `LinkedList`

More about interfaces later

- Both implementations in **`java.util`** follow the same interface **`List`**

→ Same API (application programming interface) and same methods

→ **Advantage**: both can be used interchangeably depending on memory and performance requirements

→ Changing the actual implementation is easy because only code places with the **`new`** operator must be changed

```java
import java.util.*;

class Playground {
    public static void main(String[] args) {
        List<String> words = new ArrayList<String>();
        words.add("1");
        words.add("2");
        System.out.println(words.get(0));
        System.out.println(words.contains("1"));
        for (String word : words) {
            System.out.println(word);
        }
    }
}
```

```java
import java.util.*;

class Playground {
    public static void main(String[] args) {
        List<String> words = new LinkedList<String>();
        words.add("1");
        words.add("2");
        System.out.println(words.get(0));
        System.out.println(words.contains("1"));
        for (String word : words) {
            System.out.println(word);
        }
    }
}
```

W04E02 Fruit List Not started yet.

Start exercise | in-class | bonus | Easy | Due date: end of today

10 min

4 pts

- **Problem**: manipulate fruits in a list

1. Create a **`String`** list **`fruits`** and add at least 10 different fruits
2. Iterate through all elements using the “traditional” for loop
   Hint: **`for(int i = 0; i < fruits.size(); i++)`**
3. Iterate through all elements using the enhanced for each loop
   Hint: **`for(String fruit : fruits)`**
4. Remove all fruits that contain the letter “e”

Outline

- List (part 1)
- **List (part 2)**
- Stack
- Queue

java.util.List<E>

Generic type

- The **List** **interface** represents an ordered sequence of objects

```
List
+ add(element: E): boolean
+ clear(): void
+ contains(element: E): boolean
+ get(index: int): E
+ isEmpty(): boolean
+ remove(index: int): E
+ remove(element: E): boolean
+ set(index: int, element: E): E
+ size(): int
```

Excursion: generic types

- Placeholder for class based types (**no** primitive types)
- **Type safety**: ensure to use consistent data types
- **Reusability**: write code once and use it with different data types
- Prevent the need for casting and for exceptions
  → the compiler can identify mistakes, e.g., adding **`int`** to a **`String` list**
- Make the code cleaner and easier to understand

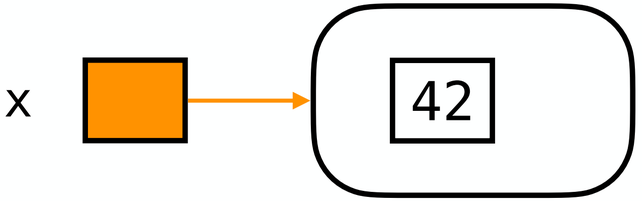
Wrapper classes for primitive types

Primitive type	**Wrapper class**
`byte`	**`Byte`**
`short`	**`Short`**
`int`	**`Integer`**
`long`	**`Long`**
`float`	**`Float`**
`double`	**`Double`**
`char`	**`Character`**
`boolean`	**`Boolean`**

Turn a primitive data type into an object

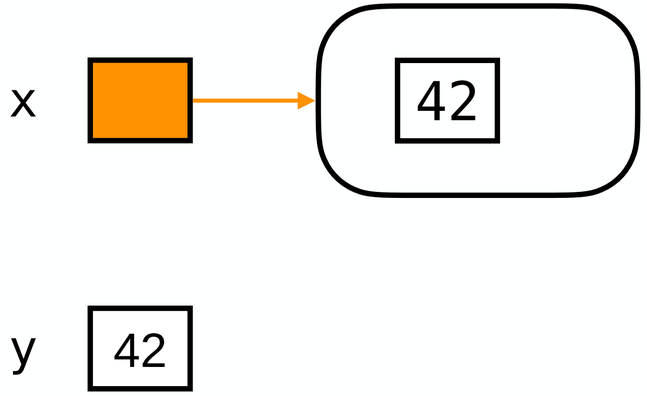
Example

```
Integer x = new Integer(42);
```



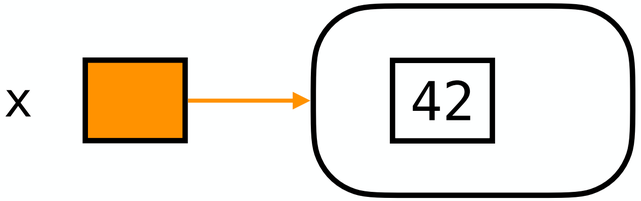
Unwrapping

- Wrapped values can also be unwrapped again
- The assignment `int y = x;` is converted automatically

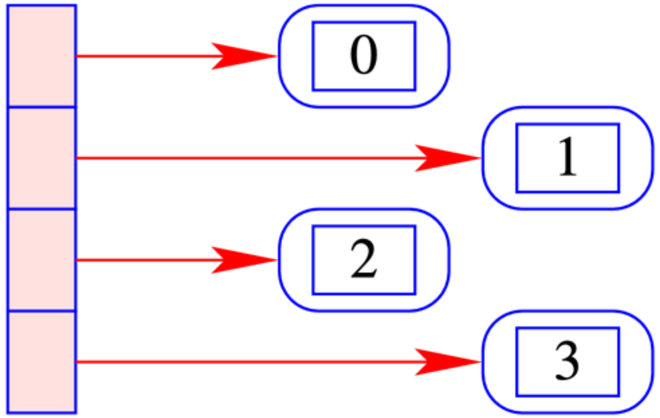


Wrapping

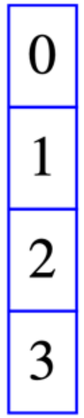
- Conversely, when an **`int`** value is assigned to an **`Integer`** variable, e.g., **`Integer x = 42;`** the constructor is called automatically



`Integer` vs. `int`



`Integer []`



`int []`

+ Integers can be stored in collection data types with generic types (e.g., **`List`**, **`Set`**)
– They need more space (~ twice as much)
– They lead to many small objects distributed over the whole memory → worse cache behavior
– You have to deal with **`null`**

The class **`Object`**

Covered later

- The **`Object`** class is a common superclass for all classes
- All classes are therefore related to **`Object`**
- Useful methods of the **`Object`** class

```
String toString()
```

returns (any) representation as **`String`**

```
boolean equals(Object obj)
```

tests for object identity or reference equality

Example

```
public boolean equals(Object obj) {
    return this == obj;
}
```

Error handling and common pitfalls

- Collections (**`Set`**, **`List`**, etc.) contain class-based data types
  - Can contain **`null`** values
  - To be safe, check for **`null`** when retrieving single elements
- Concurrent modification exceptions occur when a collection is modified while iterating over it
  - Use workarounds to modify the collection after the iteration, e.g. collecting elements to be removed in a second collection

    Covered later
  - Alternatively, use functional programming approaches such as streams to modify collections
- Some operations in collections are inefficient (e.g., adding an element at the beginning of a list) and should be avoided if possible

Example **List** with **Integer**

```java
import java.util.*;

public class Playground {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add(0);
        numbers.add(1);
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);
        numbers.add(5);
        numbers.add(8);
        numbers.add(13);
        numbers.add(21);

        System.out.println(numbers.get(5));
        System.out.println(numbers.contains(5));
        System.out.println(numbers.indexOf(5));

        int sum = 0;
        for (Integer number : numbers) {
            System.out.println(number);
            sum = sum + number;
        }
        System.out.println("Sum: " + sum);
    }
}
```

Which sequence is used here?

Example **List** with **Float**

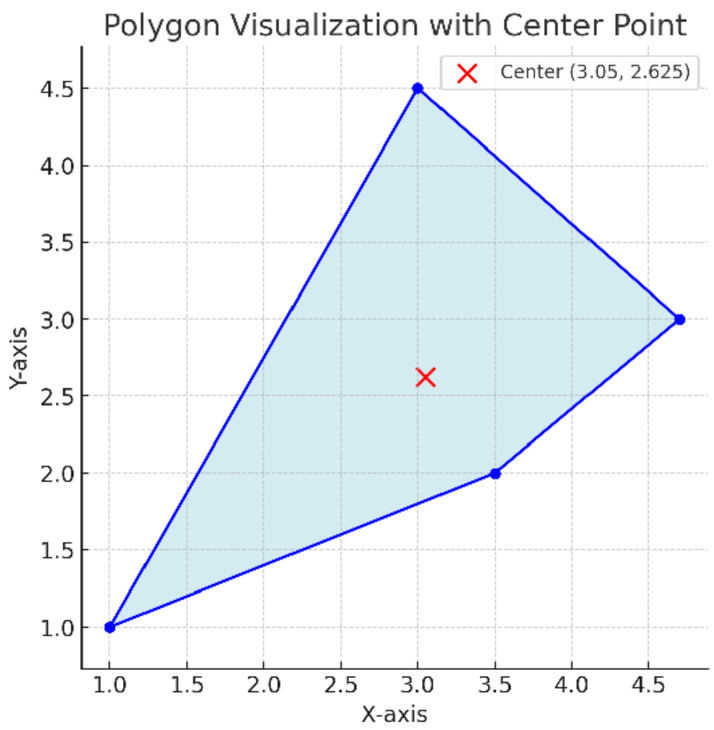
```java
import java.util.*;

public class Playground {
    public static void main(String[] args) {
        // Create a list of Float temperatures in Celsius
        List<Float> temperatures = new ArrayList<>();
        temperatures.add(18.5f);
        temperatures.add(21.0f);
        temperatures.add(19.8f);
        temperatures.add(23.4f);
        temperatures.add(20.1f);
        temperatures.add(17.9f);
        temperatures.add(22.3f);

        // Calculate the average temperature
        float sum = 0.0f;
        for (Float temp : temperatures) {
            sum = sum + temp;
        }

        float average = sum / temperatures.size();
        System.out.println("Average Temperature: " + average + "°C");
    }
}
```

Custom data types in collection data types

```java
public class Point {
    double x;
    double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return y;
    }

    public void setY(double y) {
        this.y = y;
    }
}
```

Example **List** with **Point**

```java
public class Playground {
    public static void main(String[] args) {
        // Create a list of Points to represent a polygon
        List<Point> polygon = new ArrayList<>();
        polygon.add(new Point(1, 1));
        polygon.add(new Point(3.5, 2));
        polygon.add(new Point(4.7, 3));
        polygon.add(new Point(3, 4.5));

        // Variables to store the sum of x and y coordinates
        double sumX = 0.0;
        double sumY = 0.0;

        // Enhanced for loop to calculate the sum of x and y coordinates
        for (Point point : polygon) {
            sumX = sumX + point.getX();
            sumY = sumY + point.getY();
        }

        // Calculate the average x and y coordinates
        double centerX = sumX / polygon.size();
        double centerY = sumY / polygon.size();

        // Print the center point of the polygon
        System.out.println("Center: (" + centerX + ", " + centerY + ")");
    }
}
```



Exercise

10 min

Gradebook with **`List`** to calculate averages

- Create a custom class **`Grade`** with a **`subject`** (**`String`**) and **`score`** (**`double`**)
- Initialize a **`grades`** list: use **`ArrayList<Grade>`** to store multiple grades
- Add sample grades: populate the list with sample grades (e.g., Marketing: 1.3, Economics 1: 2.0)
- Calculate average: use a for loop to compute the average score
- Display results: print each grade and the calculated average
- **Optional challenge:** add **credits** (**`int`**) to **`Grade`** and take it into account as weight of the grade

Outline

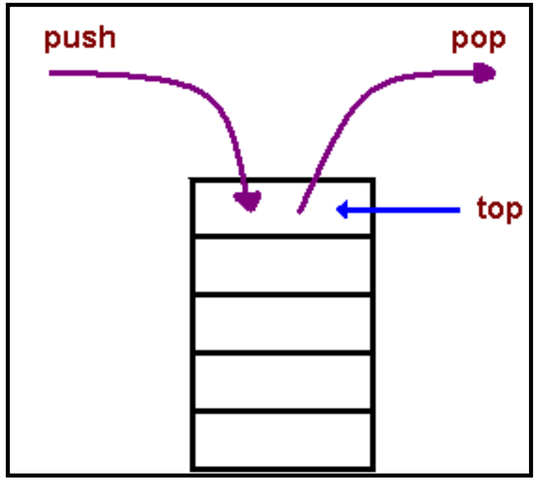
- List (part 1)
- List (part 2)
- **Stack**
- Queue

Stack

- Operations

`void push(Object element)`	puts **`element`** on top of the stack
`Object pop()`	returns top **`element`**
`boolean isEmpty()`	tests for emptiness
`String toString()`	returns a string representation

- Follows the LIFO principle: last in, first out



Friedrich Ludwig Bauer, TUM

Modeling a **`Stack`**

```
Stack
---------------------------------
+ push(element: Object): void
+ pop(): Object
+ isEmpty(): boolean
+ toString(): String
```

`Stack` visualization

```
stack.push(1);
```

Size: 0

stack

`Stack` visualization

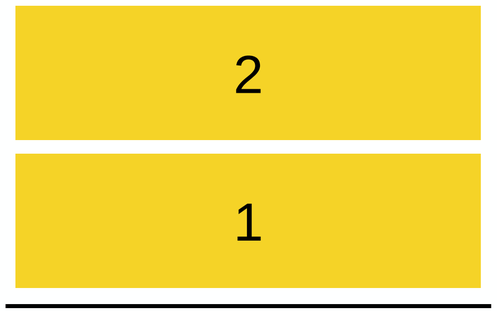
```
stack.push(2);
```

Size: 1

stack

`Stack` visualization

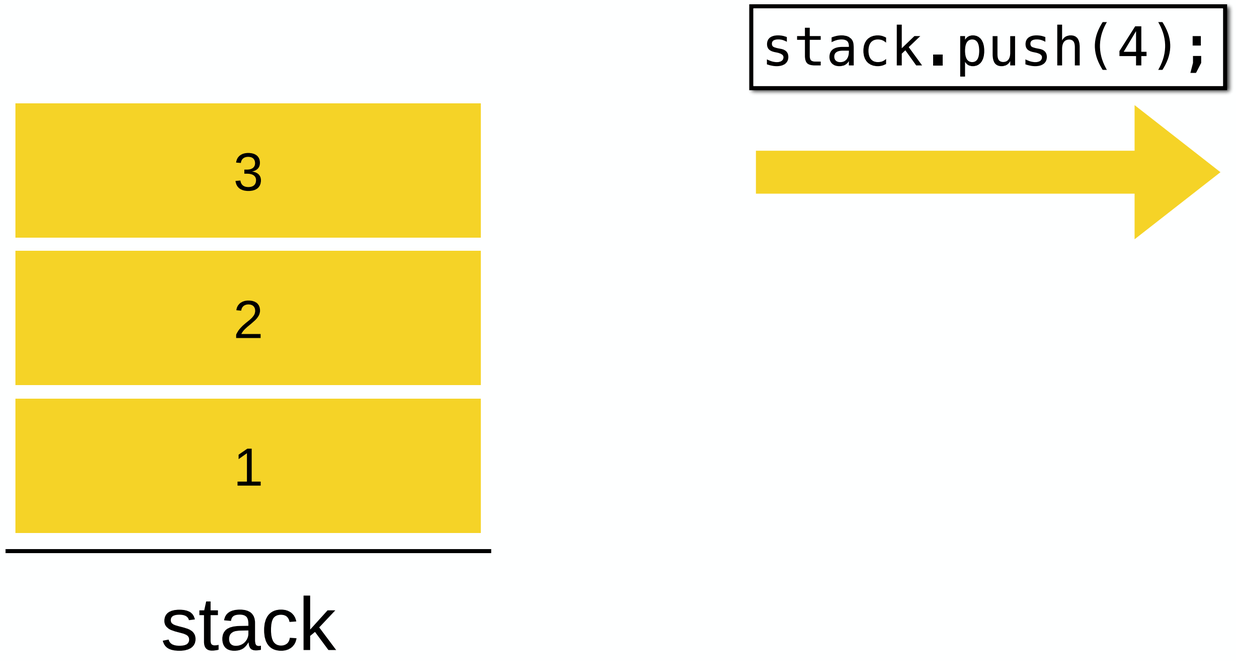
```
stack.push(3);
```



stack

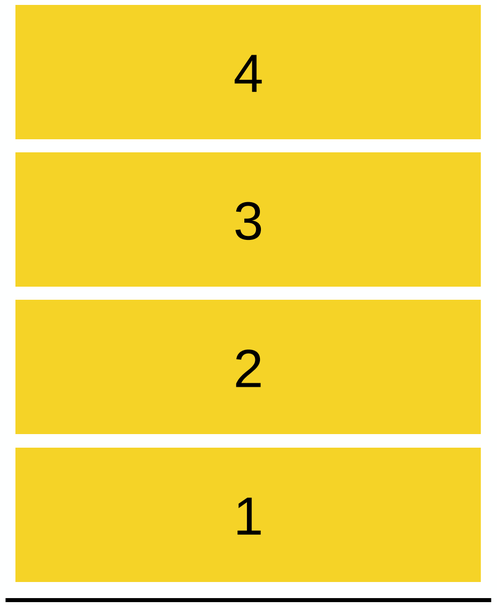
Size: 2

`Stack` visualization



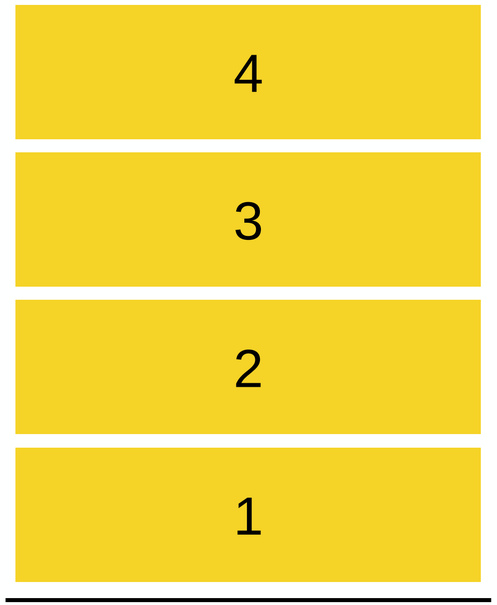
Size: 3

`Stack` visualization



Size: 4

`Stack` visualization



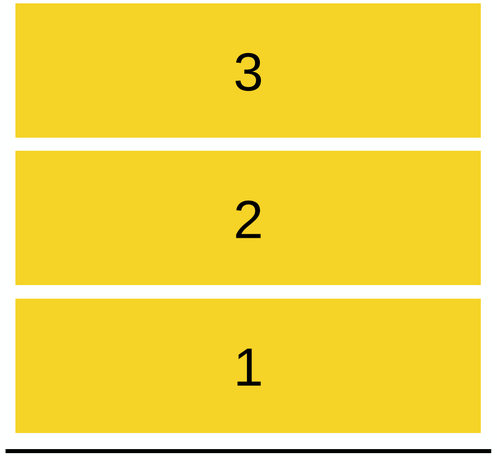
stack

Size: 4

```
stack.pop();
```

returns 4

`Stack` visualization



stack

Size: 3

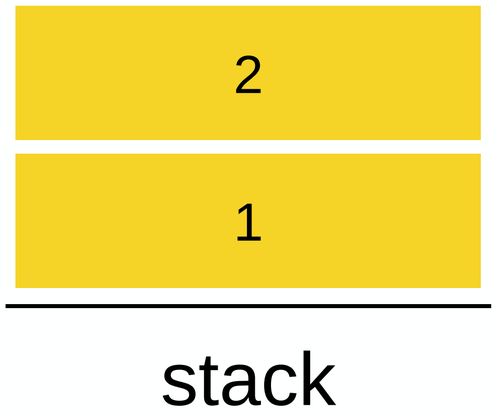
```
stack.pop();
```

returns 3

`Stack` visualization

returns 2

```
stack.pop();
```



Size: 2

`Stack` visualization

returns 1

```
stack.pop();
```

Size: 1

stack

`Stack` visualization

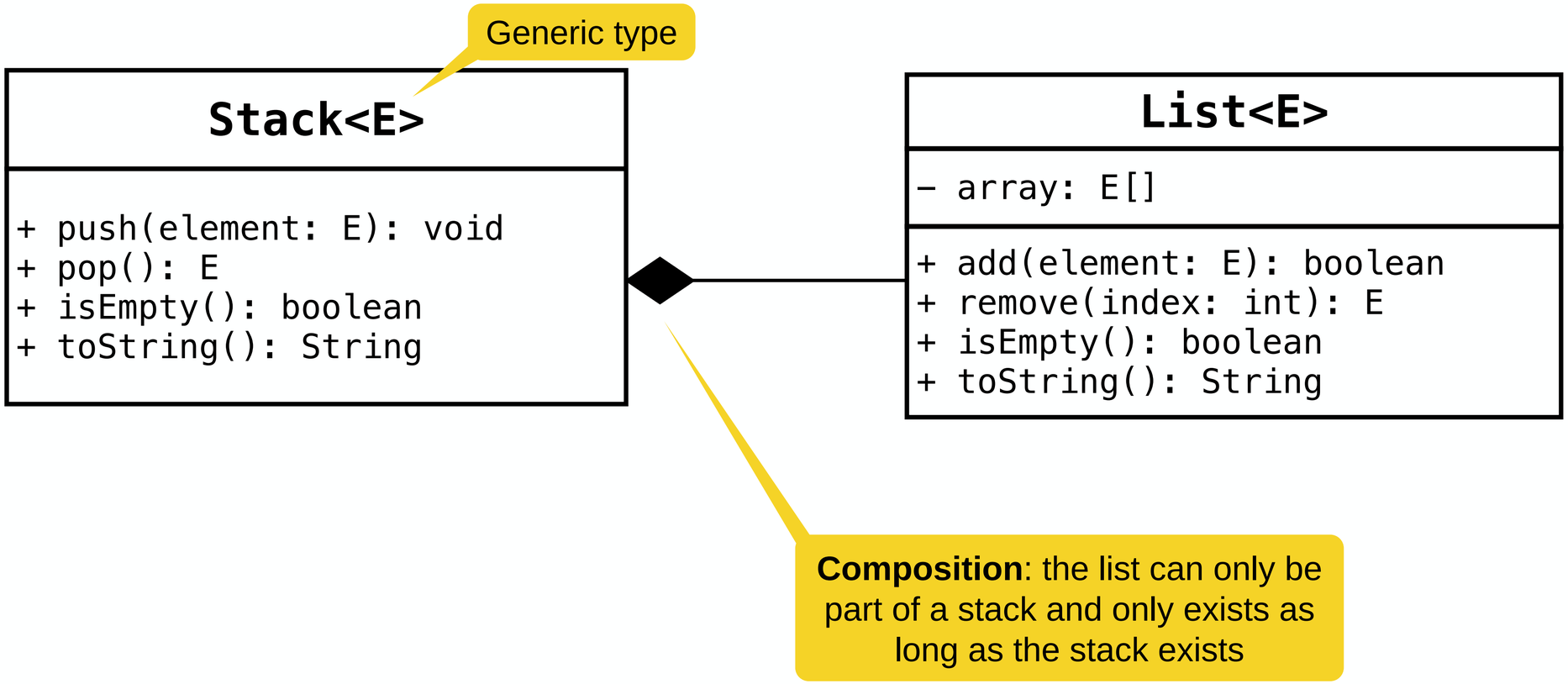
returns **`null`** in our implementation

```
stack.pop();
```

Size: 0

stack

Modeling a **Stack** with a **List**



Implementation: **`Stack`** with a **List**

```java
import org.checkerframework.checker.nullness.qual.*;
import java.util.*;

public class Stack<E> {

    private final List<E> list = new ArrayList<>();

    public void push(@NonNull E element) {
        list.add(element);
    }

    @Nullable
    public E pop() {
        if (isEmpty()) {
            return null;
        }
        return list.remove(list.size() - 1);
    }

    public boolean isEmpty() {
        return list.isEmpty();
    }

    public String toString() {
        return list.toString();
    }
}
```

Add to the end of the list

Remove the last element

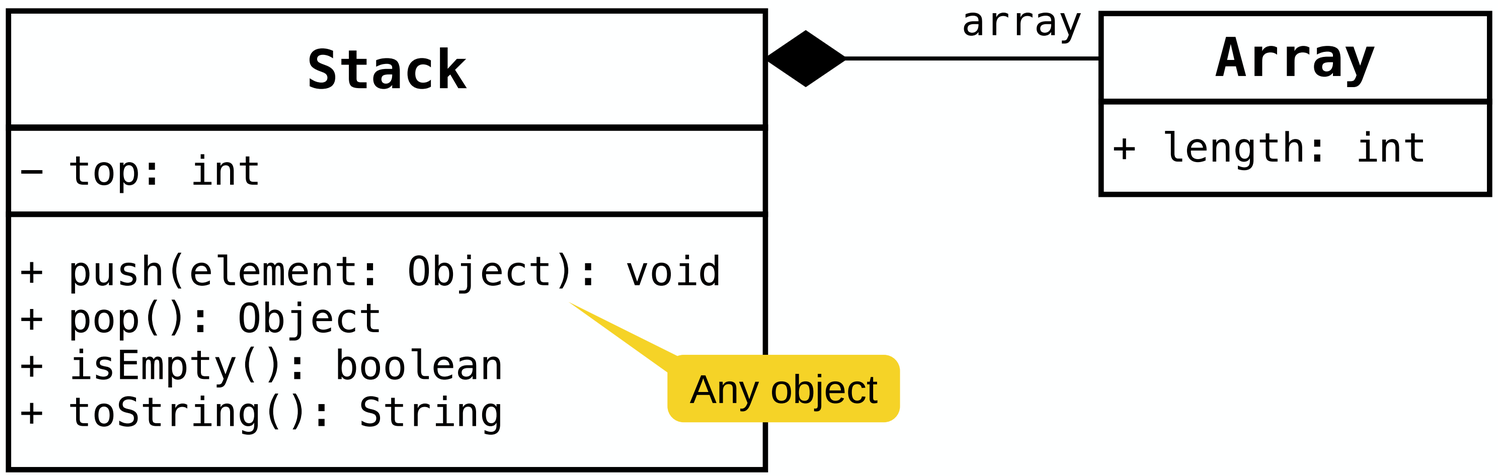
- Simple implementation
- Does not use all features of **`List`**

- Second idea
  - Realize the stack directly with an array and a pointer to the top occupied cell
  - If the array overflows, we replace it with a larger one

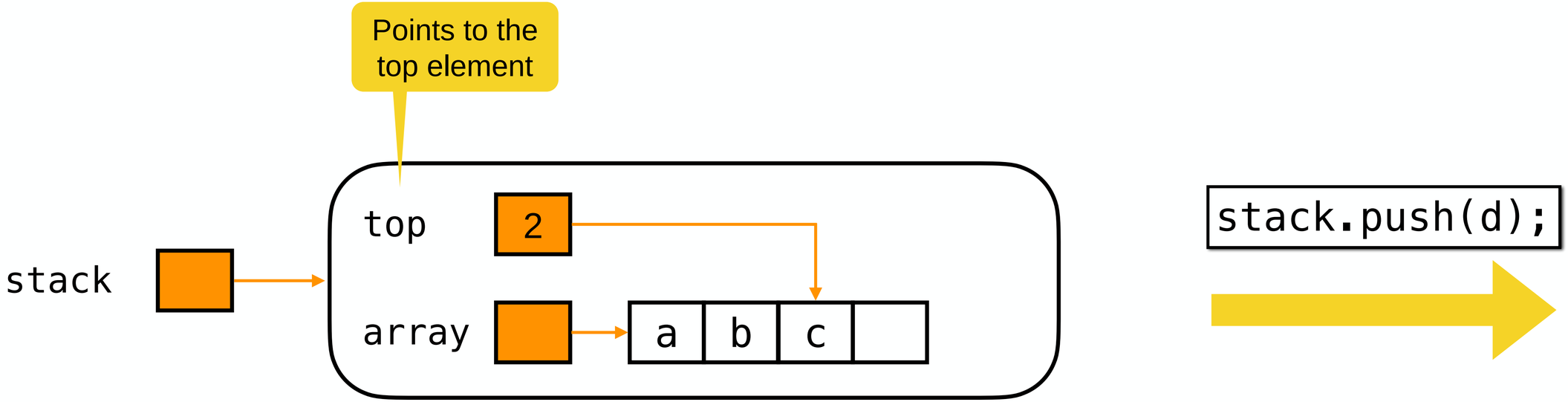
Example usage of **Stack** with a **List**

```java
public class Main {
    public static void main(String[] args) {
        // Create a stack of integers
        Stack<Integer> stack = new Stack<>();

        // Push elements onto the stack
        System.out.println("Push elements onto the stack:");
        stack.push(10);
        stack.push(20);
        stack.push(30);
        System.out.println("Current stack: " + stack);

        // Pop elements from the stack
        System.out.println("Pop elements from the stack:");
        Integer poppedElement1 = stack.pop();
        System.out.println("Popped element: " + poppedElement1);
        System.out.println("Current stack: " + stack);
    }
}
```

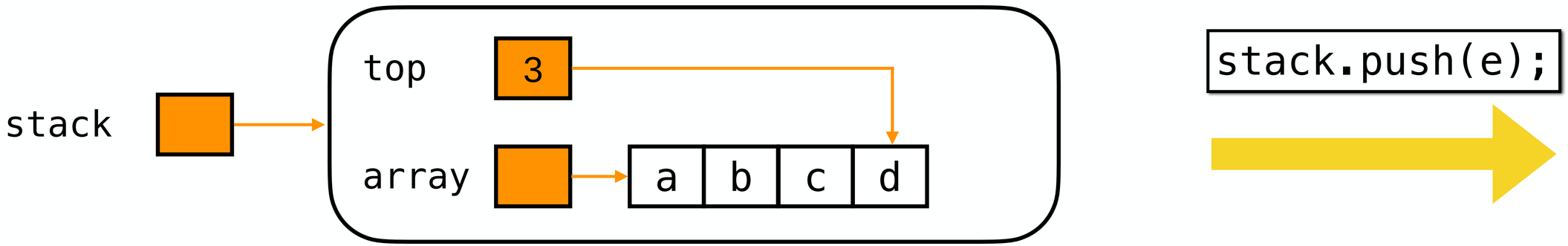
Modeling a **`Stack`** with an **`Array`**



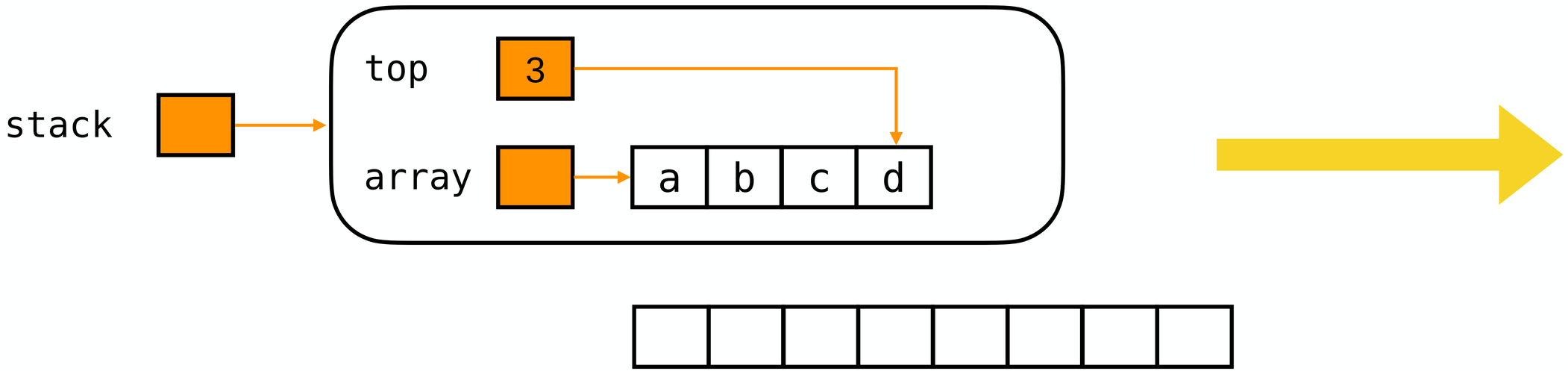
Example process: **`Stack`** with an **`Array`**



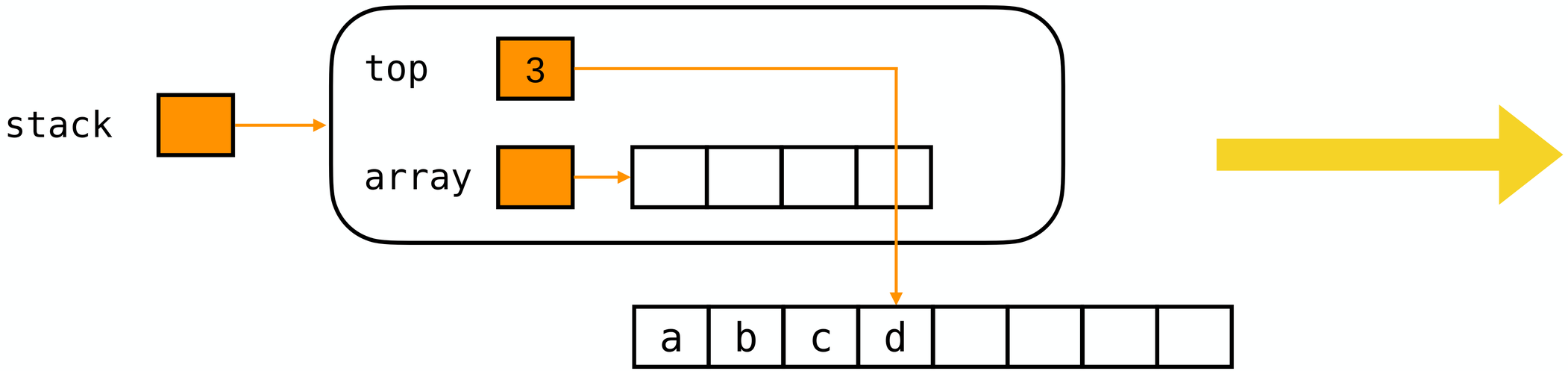
Example process: **`Stack`** with an **`Array`**



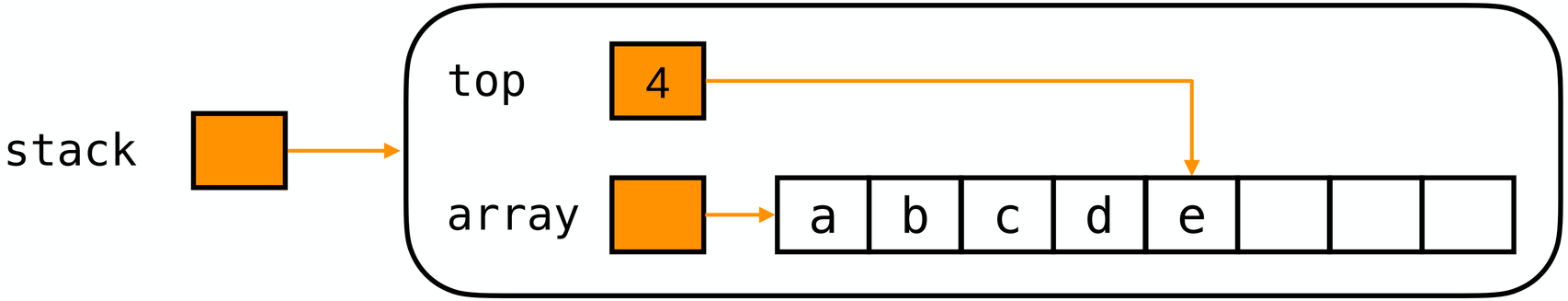
Example process: **`Stack`** with an **`Array`**



Example process: **`Stack`** with an **`Array`**



Example process: **`Stack`** with an **`Array`**



Implementation: **Stack** with an **Array**

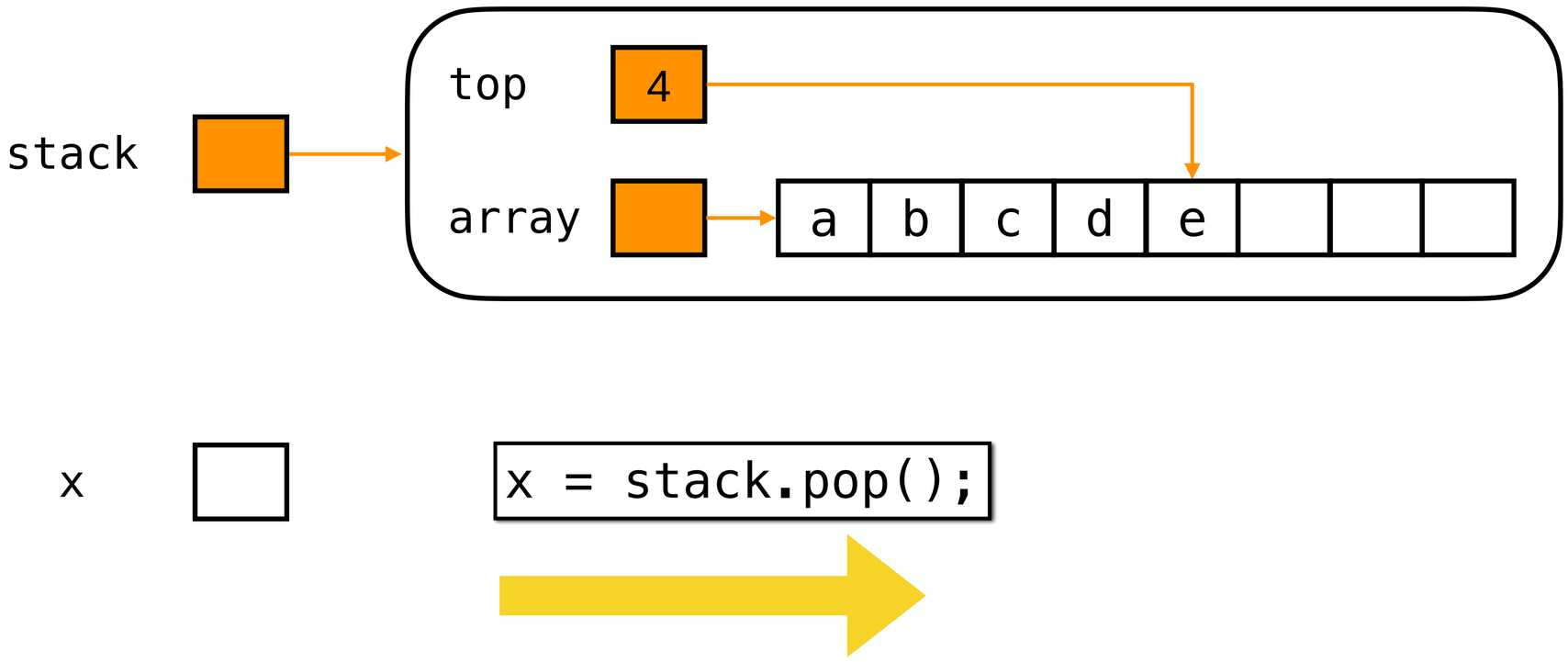
```java
import java.util.*;

public class Stack {
    private int top = -1;
    private Object[] array = new Object[4];

    public boolean isEmpty() {
        return top < 0;
    }
    public void push(Object element) {
        top++;
        if (top == array.length) {
            Object[] newArray = new Object[2 * top];
            for(int i = 0; i < top; i++) {
                newArray[i] = array[i];
            }
            array = newArray;
        }
        array[top] = element;
    }
    public Object pop() {    // Assumption top > -1
        return array[top--];
    }
    public String toString() {
        return Arrays.toString(
            Arrays.copyOfRange(array, 0, top + 1));
    }
}
```

- **Disadvantage**: new space is allocated but never released
- The implementation is not type safe
- **Idea:** if the length drops to half again, we release it

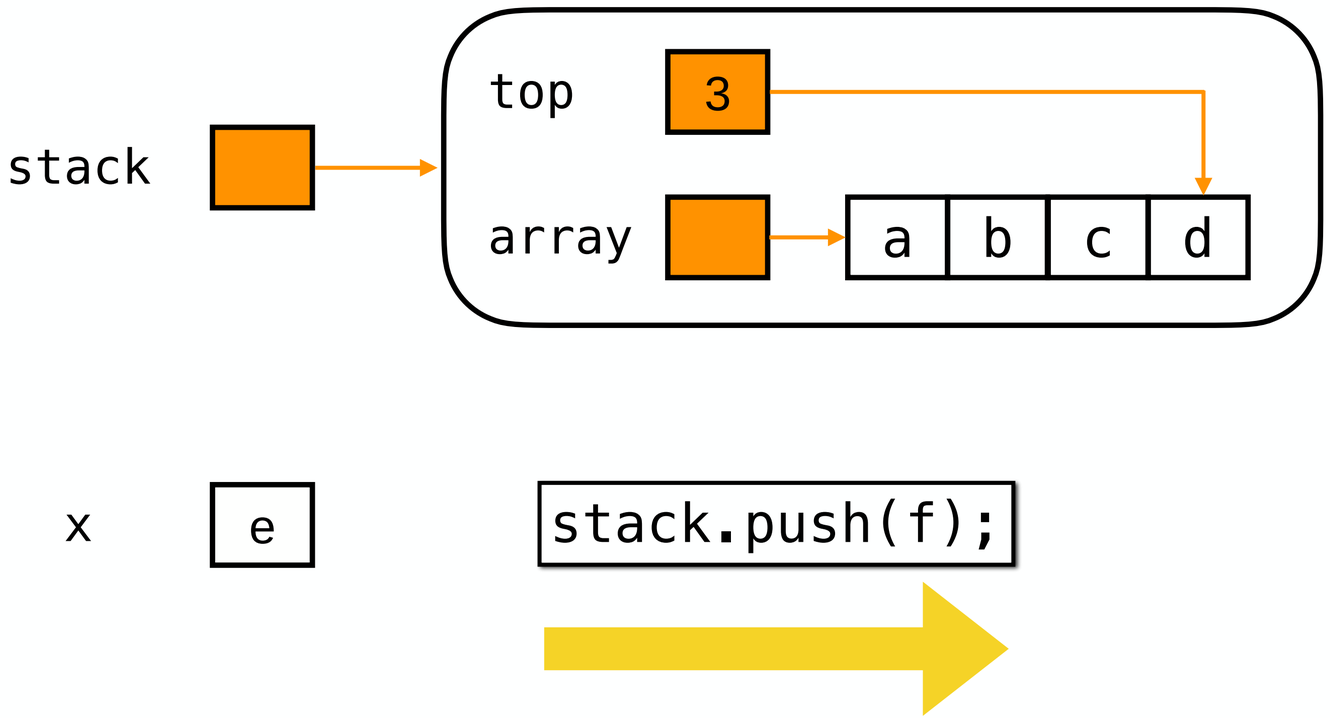
Double the array size, copy all existing elements into a new array and use this one from now on

Only include elements that are currently in the Stack (simplified, **inefficient** version)

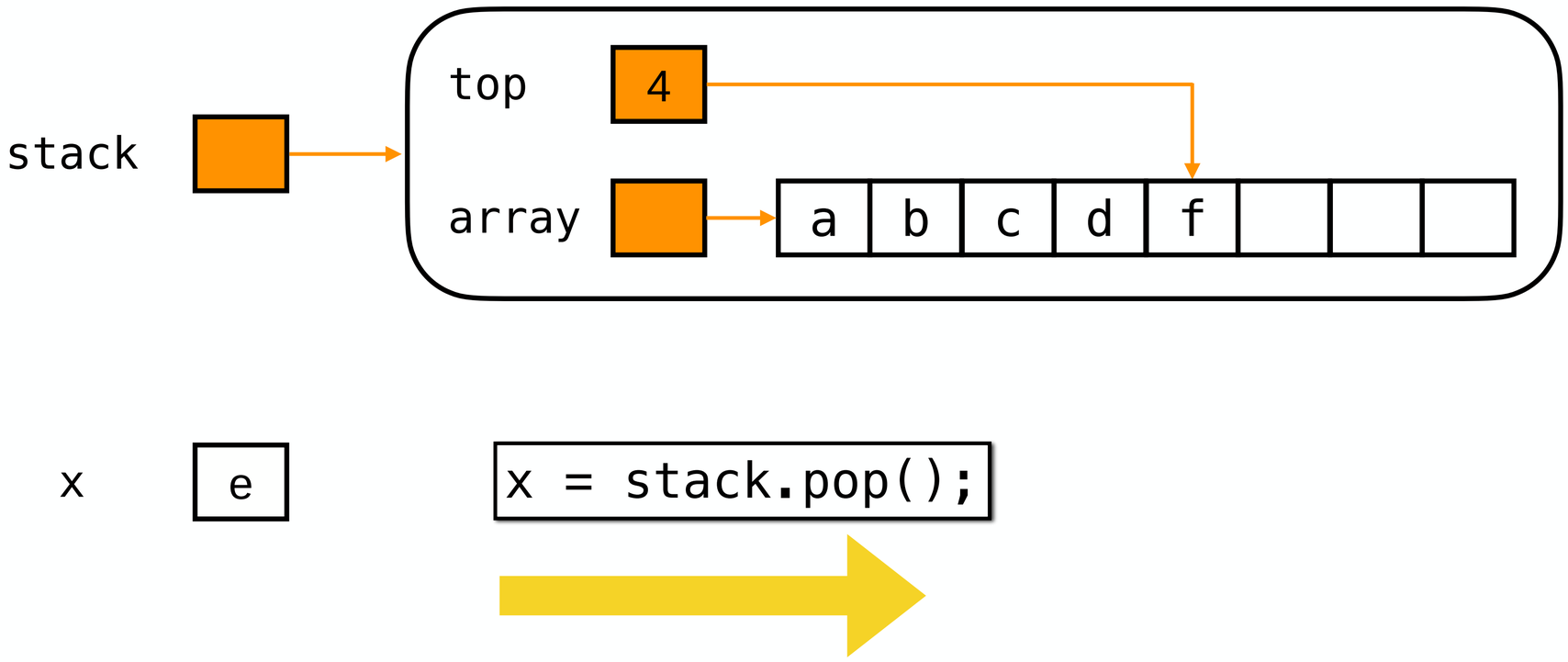
Example process: **`Stack`** with an **`Array`**



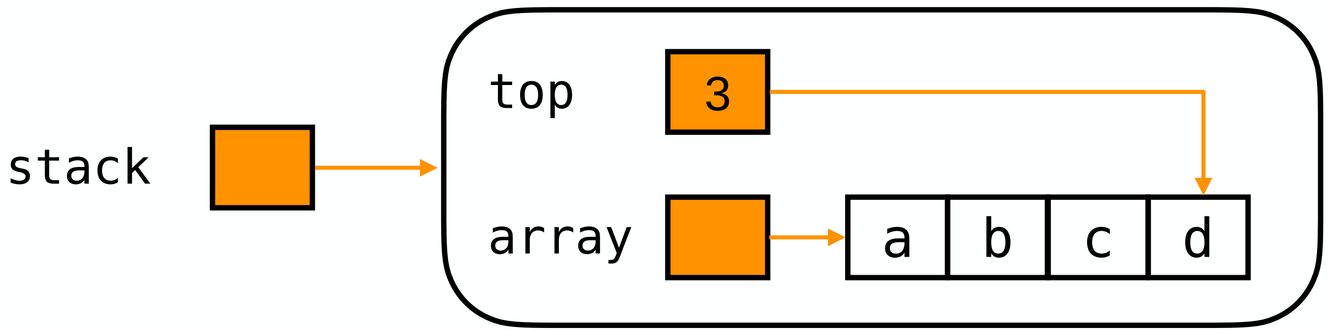
Example process: **`Stack`** with an **`Array`**



Example process: **`Stack`** with an **`Array`**



Example process: **`Stack`** with an **`Array`**

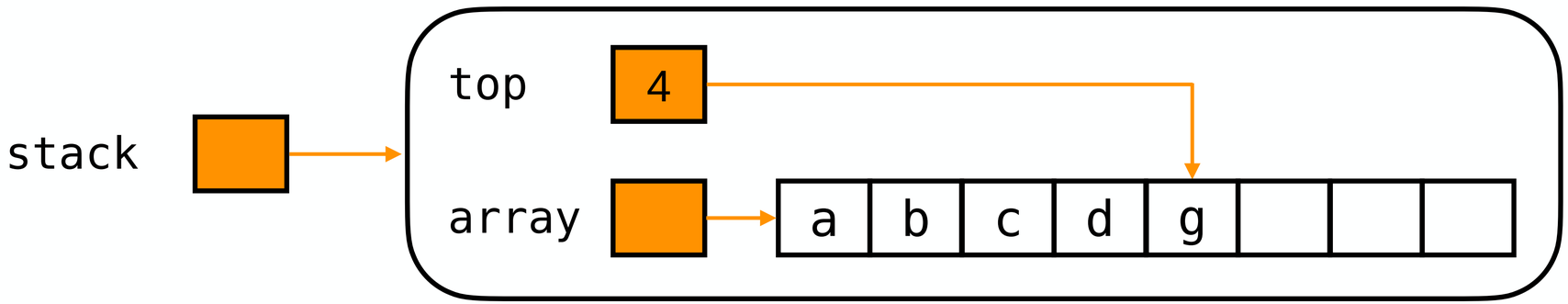


x

f

```
stack.push(g);
```

Example process: **`Stack`** with an **`Array`**



x

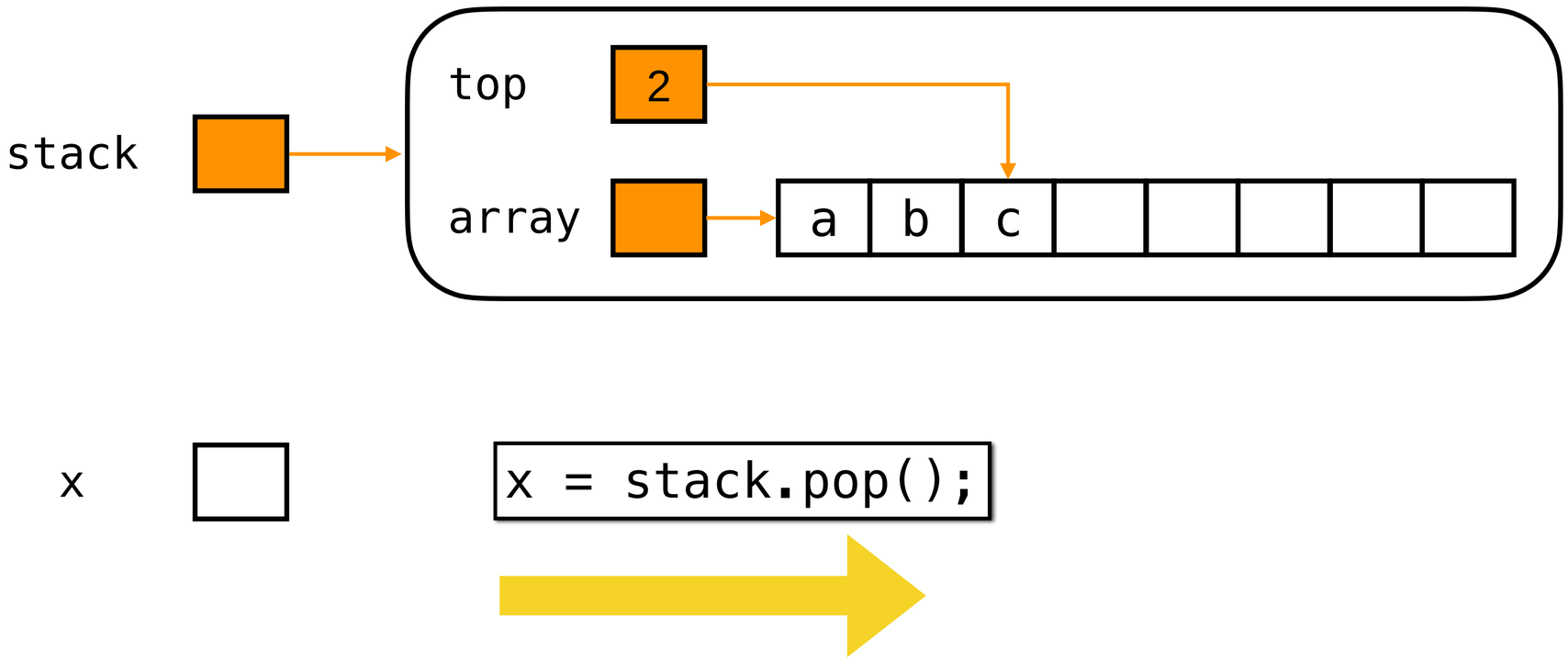
f

```
x = stack.pop();
```

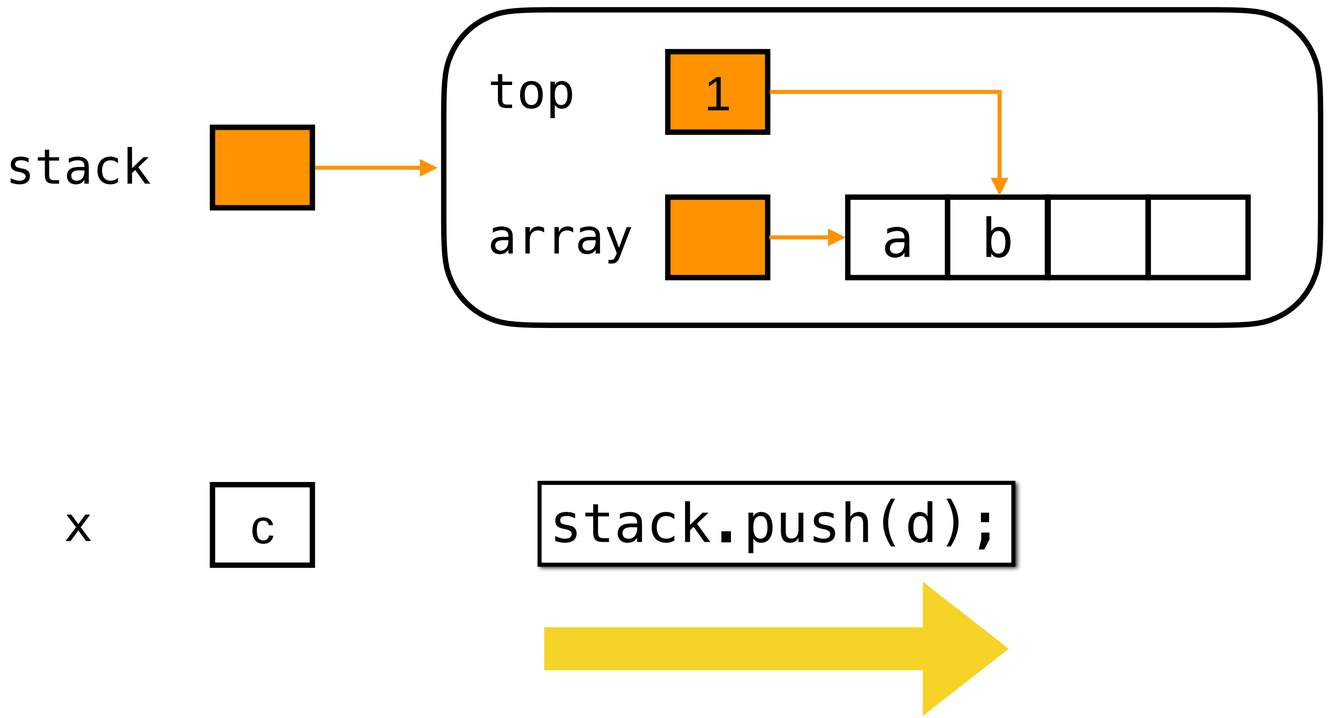
Observation

- In the worst case, all elements must be copied for each operation
- Idea: the stack only releases when the length drops to a quarter - and then only half

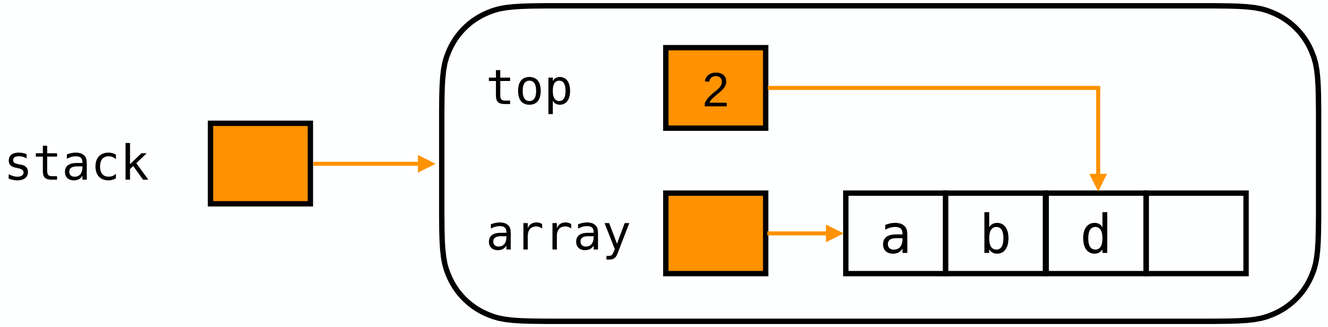
Example process: **`Stack`** with an **`Array`**



Example process: **`Stack`** with an **`Array`**

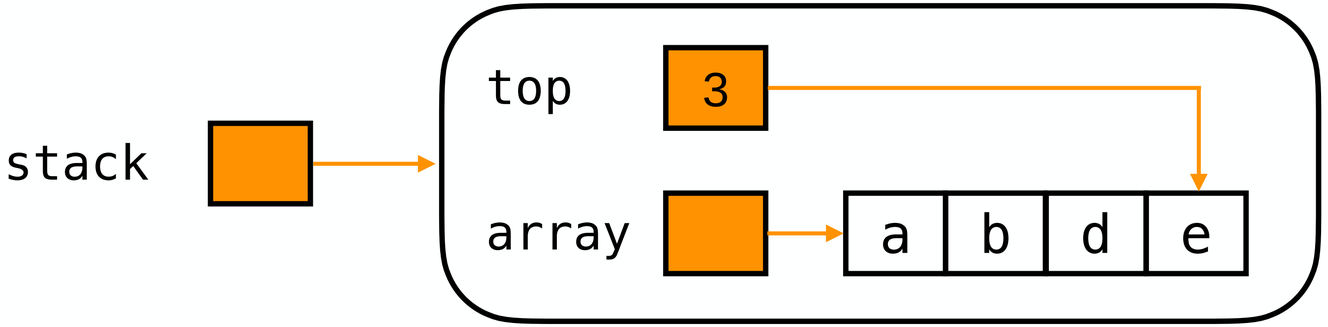


Example process: **`Stack`** with an **`Array`**




```
stack.push(e);
```

Example process: **`Stack`** with an **`Array`**



Analysis

- Before each copy, at least half as many operations are performed as elements are copied
- Averaged over the entire sequence of operations, a maximum of two numbers are copied per operation (amortized effort analysis)

```
public Object pop() {
    // Assumption top > -1
    Object result = array[top];
    if (top == array.length / 4 && top >= 2) {
        Object[] newArray = new Object[2 * top];
        for(int i = 0; i < top; i++) {
            newArray[i] = array[i];
        }
        array = newArray;
    }
    top--;
    return result;
}
```

Covered in the course
Fundamentals of Algorithms and Data Structures

Example usage of **Stack** with an **Array**

```java
public class Main {
    public static void main(String[] args) {
        Stack stack = new Stack();

        // Push elements onto the stack
        stack.push("A");
        stack.push("B");
        stack.push("C");
        System.out.println("Stack after pushes: " + stack);

        // Pop an element
        System.out.println("Popped element: " + stack.pop());
        System.out.println("Stack after pop: " + stack);

        // Check if stack is empty
        System.out.println("Is stack empty? " + stack.isEmpty());
    }
}
```

Note: you can copy the Stack code from https://gist.github.com/krusche/9cca72c2a8f9395897a0bae854b7c751

W04E03 Binary Conversion

Not started yet.

Start exercise | in-class | bonus | Easy | Due date: end of today

10 min

3 pts

- **Problem**: convert a positive integer number into its binary representation
  - Read an integer from the console
  - Print the binary representation to the console
  - You have to use the **`Stack`** class (based on **`java.util.List`**)
  - Example input 8 would lead to 1000, input 9 would lead to 1001, etc.

```
public static void convertNumberToBinary() {
    int number = InputReader.readInt("Enter the number: ");
    Stack<Integer> stack = new Stack<>();

    // TODO: implement

}
```

Outline

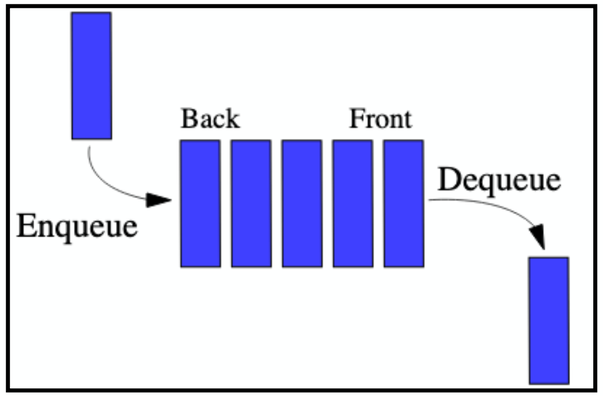
- List (part 1)
- List (part 2)
- Stack
- **Queue**

Queue operations

- (Waiting) queues manage their elements according to the **FIFO** principle: First In First Out
  - Stacks on the other hand use the **LIFO** principle: Last In First Out
- Operations

`void enqueue(Object element)`	adds the **`element`** to the queue
`Object dequeue()`	returns the **first** element
`boolean isEmpty()`	tests for emptiness
`String toString()`	returns a string representation

- Ability to create an empty queue



Modeling a **Queue**

```
Queue
+ enqueue(element: Object): void
+ dequeue(): Object
+ isEmpty(): boolean
+ toString(): String
```

`Queue` visualization

queue

```
queue.enqueue(1);
```

Size: 0

`Queue` visualization

queue | 1

```
queue.enqueue(2);
```

Size: 1

`Queue` visualization

queue | 1 | 2

```
queue.enqueue(3);
```

Size: 2

`Queue` visualization

queue | 1 | 2 | 3

```
queue.enqueue(4);
```

Size: 3

`Queue` visualization

queue

1	2	3	4

Size: 4

`Queue` visualization

Size: 4

`Queue` visualization

returns 2

```
queue.dequeue();
```

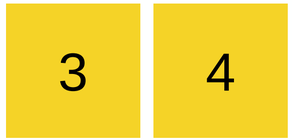
queue

Size: 3

`Queue` visualization

returns 3

queue




```
queue.dequeue();
```

Size: 2

`Queue` visualization

returns 4

queue

4

```
queue.dequeue();
```

Size: 1

Queue visualization

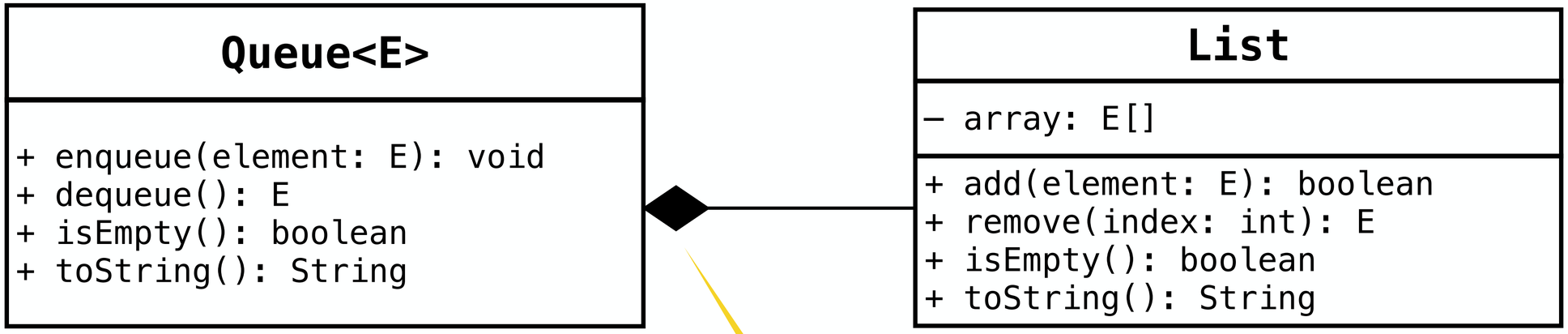
returns **null**

```
queue.dequeue();
```

queue

Size: 0

Modeling a **`Queue`** with a **`List`**



Composition: the list can only be part of a queue and only exists as long as the queue exists

Implementation: **`Queue`** with a **`List`**

```java
import org.checkerframework.checker.nullness.qual.*;
import java.util.*;

public class Queue<E> {

    @NonNull
    private final List<E> list = new ArrayList<>();

    public void enqueue(@NonNull E item) {
        list.add(item);
    }

    @Nullable
    public E dequeue() {
        if (isEmpty()) {
            return null;
        }
        return list.remove(0);
    }

    public boolean isEmpty() {
        return list.isEmpty();
    }

    public String toString() {
        return list.toString();
    }
}
```

Add to the end of the list

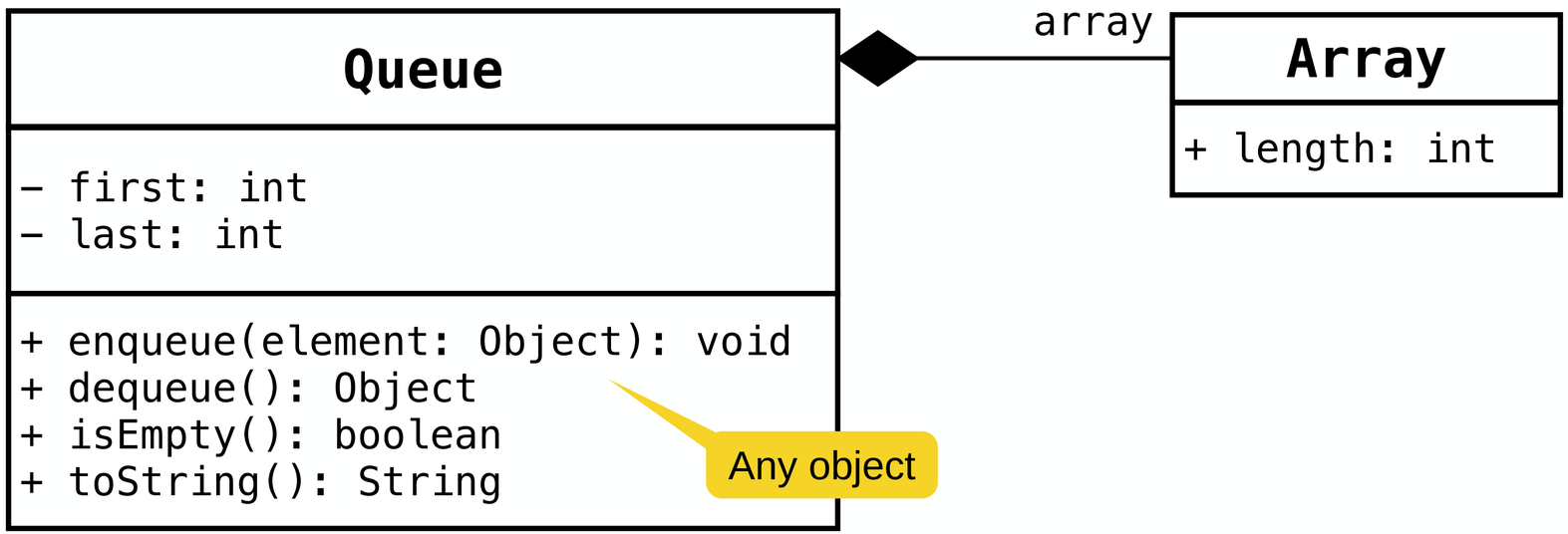
Remove the first element

- Analysis
  - Simple implementation
  - Does not use all features of **`List`**
- Second idea
  - Realize the queue directly using an array
  - If the array overflows, we replace it with a larger one

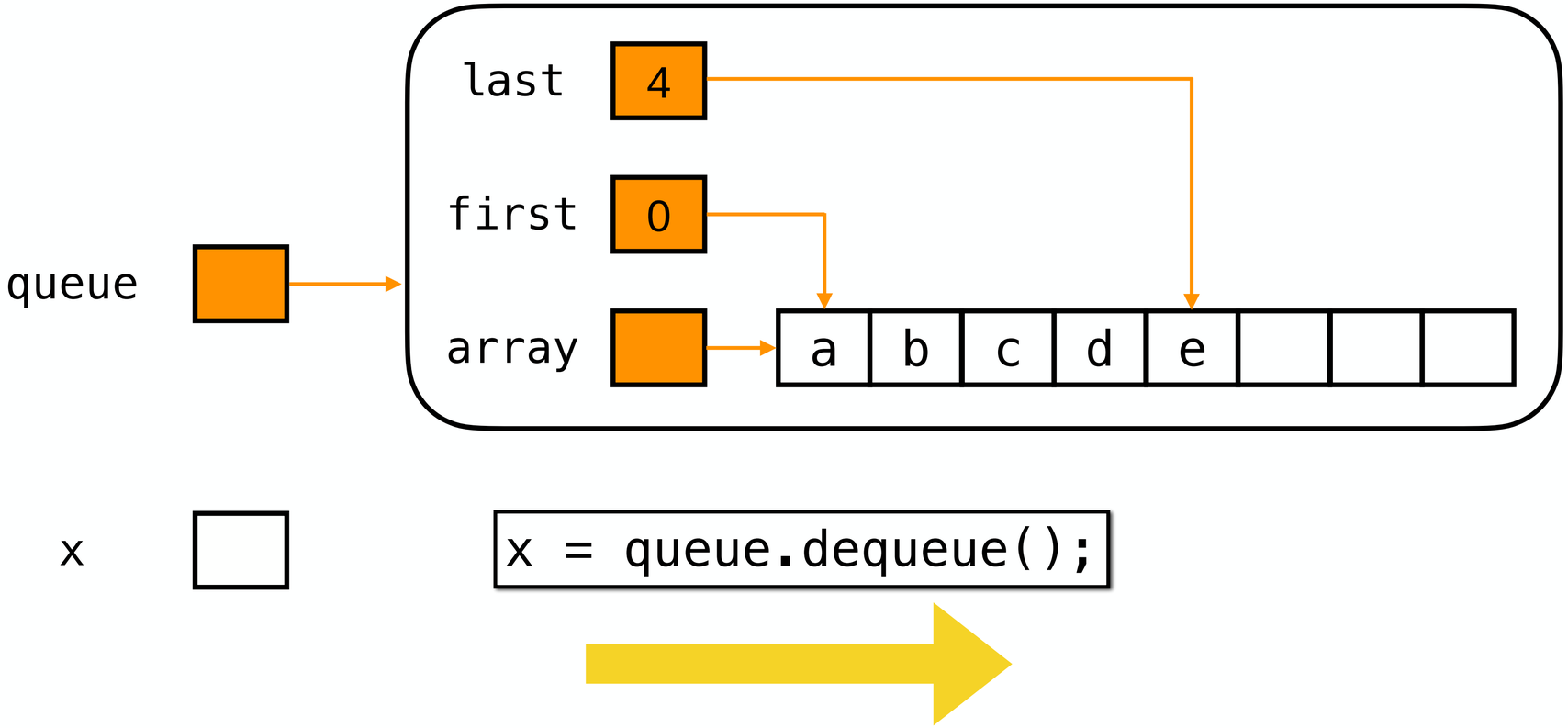
Example usage of **Queue** with **List**

```java
public class Main {
    public static void main(String[] args) {
        Queue<String> queue = new Queue<>();

        // Enqueue elements
        queue.enqueue("First");
        queue.enqueue("Second");
        queue.enqueue("Third");
        System.out.println("Queue after enqueues: " + queue);

        // Dequeue an element
        System.out.println("Dequeued element: " + queue.dequeue());
        System.out.println("Queue after dequeue: " + queue);

        // Check if the queue is empty
        System.out.println("Is queue empty? " + queue.isEmpty());
    }
}
```

Modeling a **Queue** with an **Array**

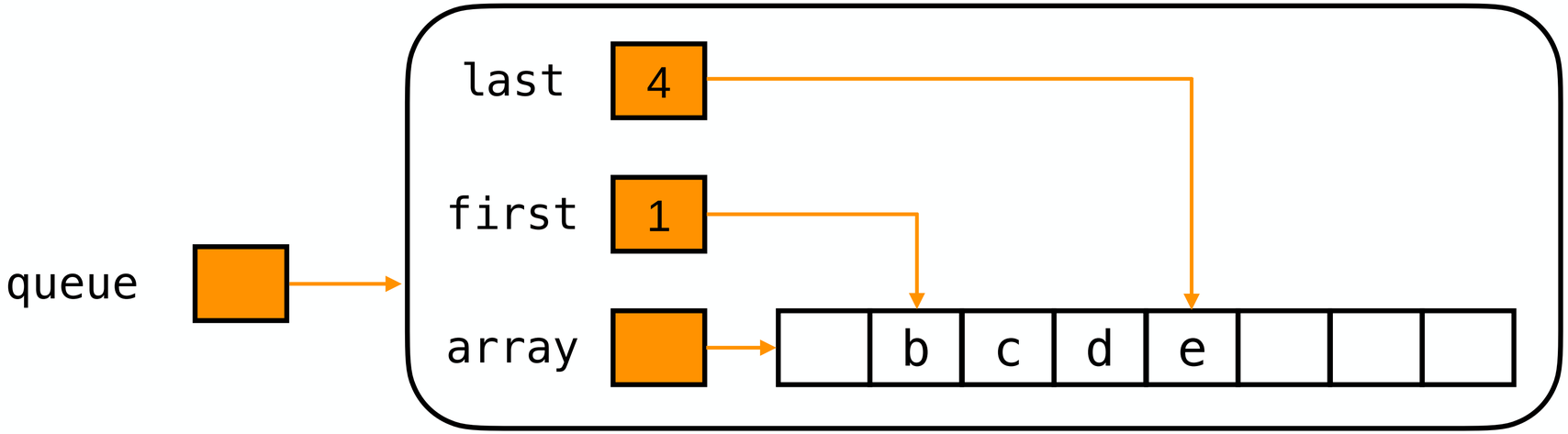


Example process: **`Queue`** with an **`Array`**



- When removing an element, **`first`** is moved one index to the right
- If **`first == last`** evaluates to **`true`**, the queue will be empty after **`dequeue()`** and we set **`first = last = -1`**

Example process: **`Queue`** with an **`Array`**

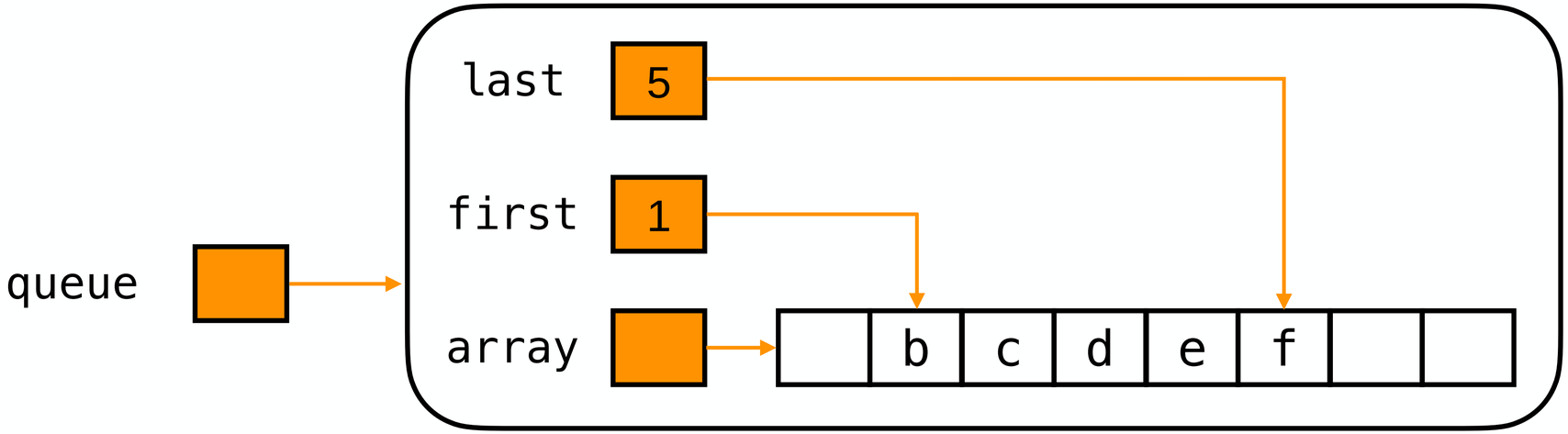


x

a

```
queue.enqueue(f);
```

Example process: **`Queue`** with an **`Array`**



- When adding an element, the value is inserted at **`array[++last]`**

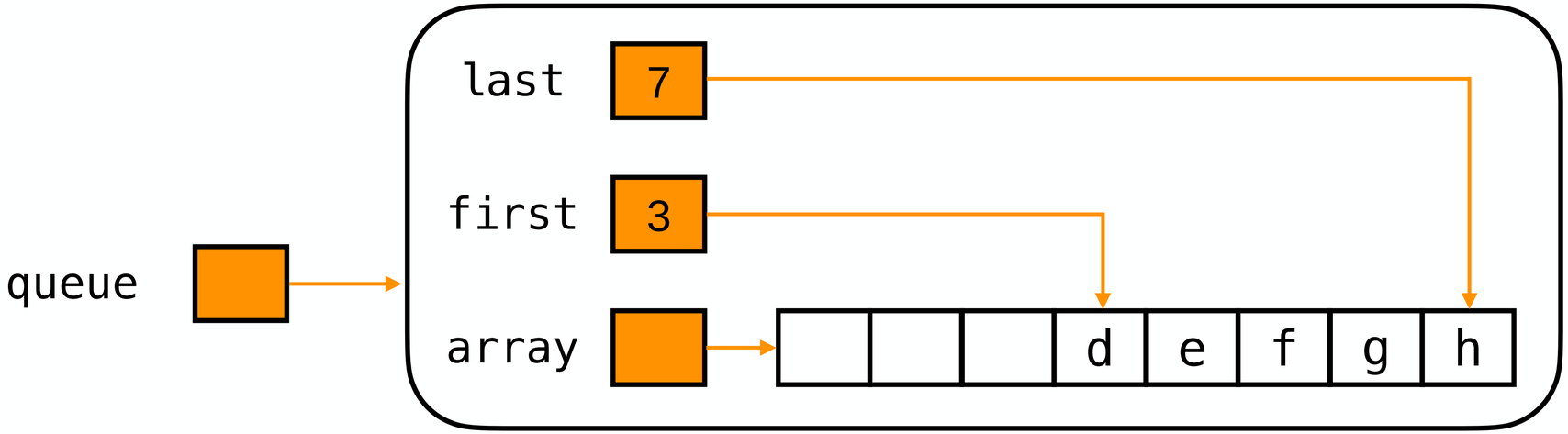
Queue: array boundaries for `enqueue()`

- What happens if `last` reaches the end of the array so that `last == array.length` is `true`?
- **First idea:** double the array size (we also do this when the array is really full)
- **Disadvantage**: all elements in the left of `first` (if there are any) will never be able to be filled again

→ First reuse elements in the left of `first`

- If there is still space at the beginning of the array (i.e. from index 0), use these cells first: jump from `array.length - 1` to `0` (using the modulo operator `%`)
- A similar phenomenon with `dequeue()`: right shift of `first` when the array boundary is already reached

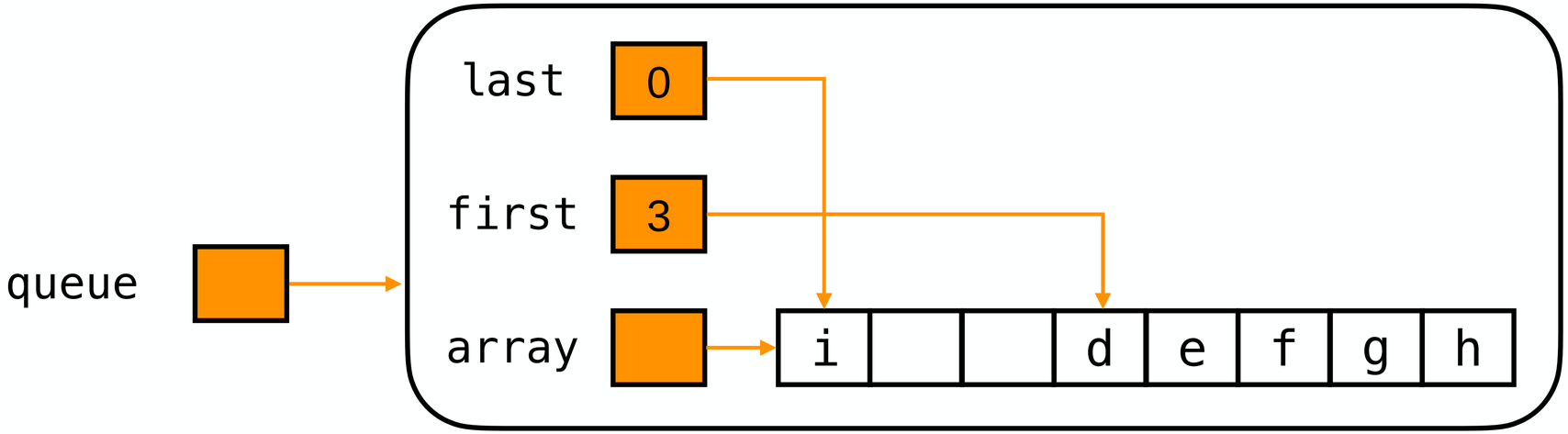
Example process: **`enqueue()`** reaching the end of the **`Array`**




```
queue.enqueue(i);
```

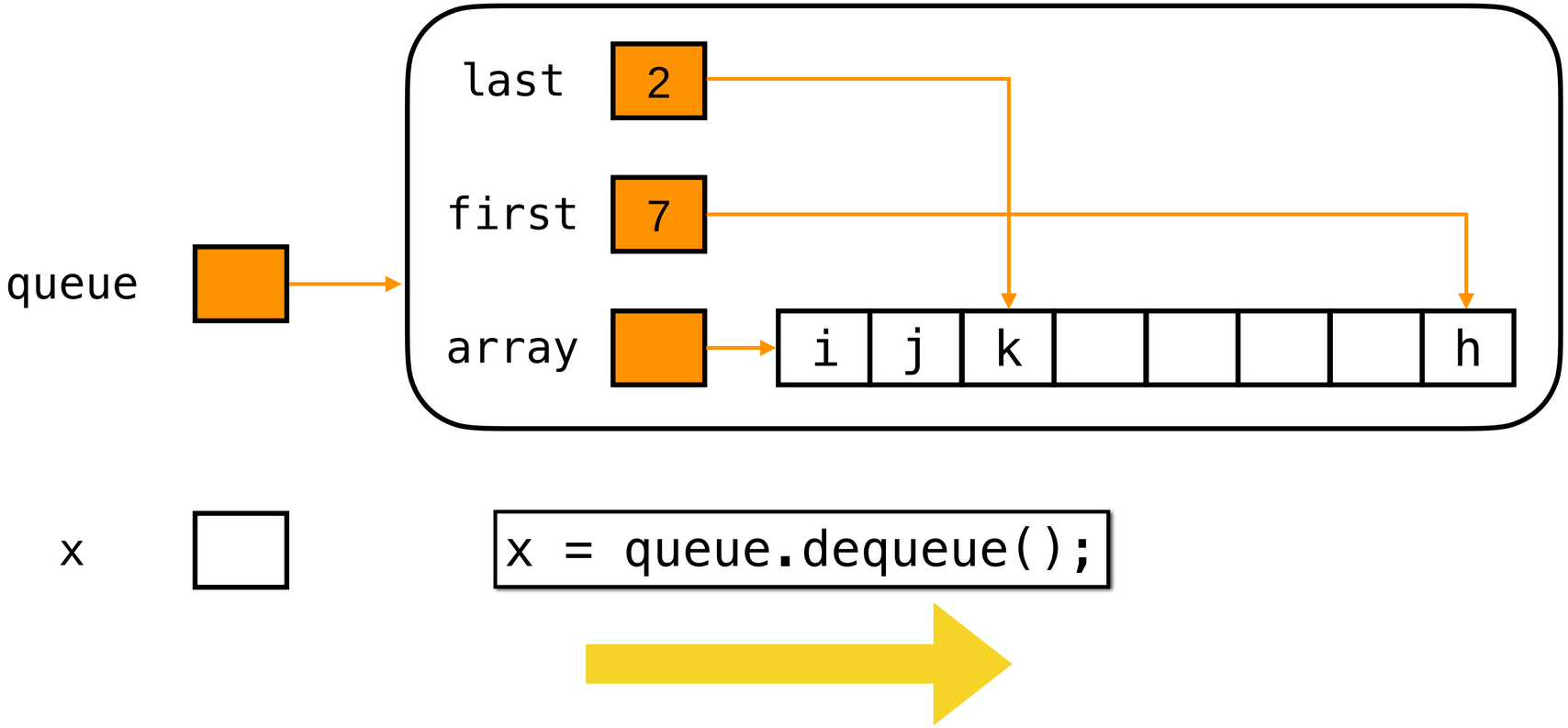
Assuming unused elements at the beginning of the array

Example process: **`enqueue()`** reaching the end of the **`Array`**

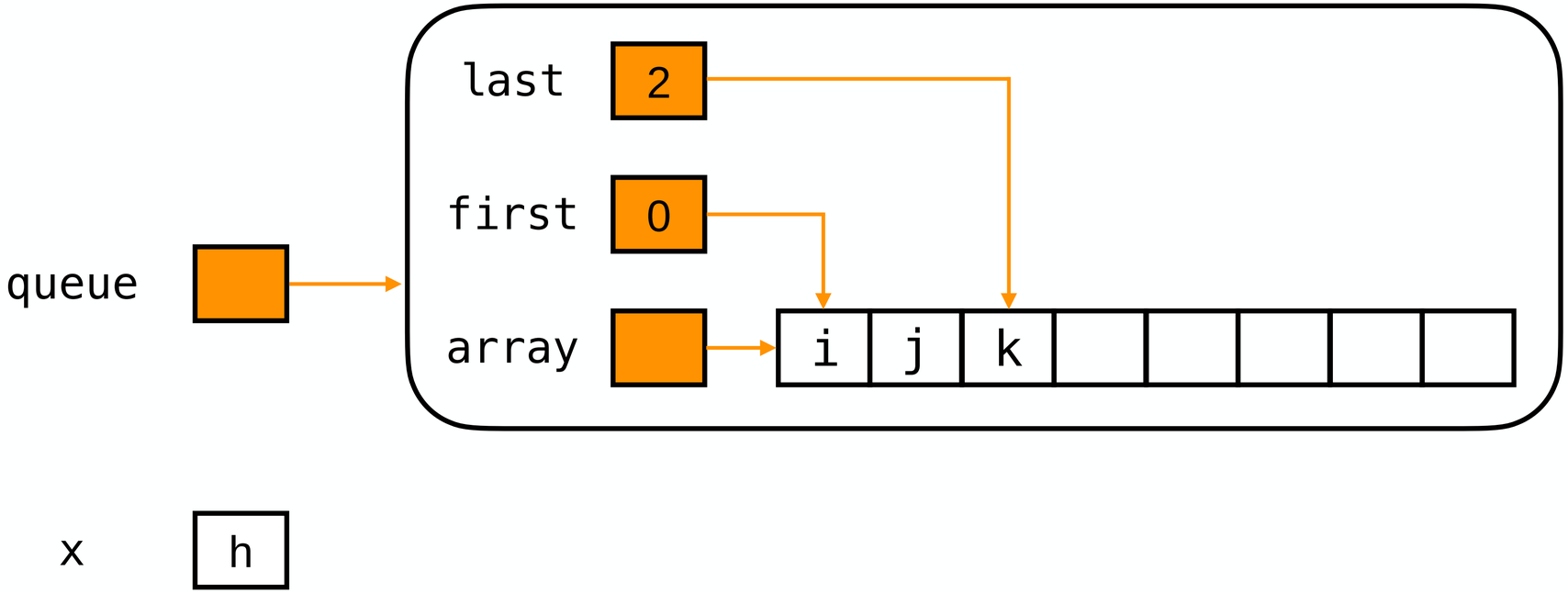


Assuming unused elements at the beginning of the array

Example process: **`dequeue()`** reaching the end of the **`Array`**



Example process: **`dequeue()`** reaching the end of the **`Array`**



Implementation: **Queue** with an **Array**

```java
import java.util.Arrays;

public class Queue {
    private int first = -1, last = -1;
    private Object[] array = new Object[4];

    // ...

    public boolean isEmpty() {
        return first == -1;
    }

    public String toString() {
        return Arrays.toString(array);
    }
}
```

Index of the first and the last element

Simplified version, will be improved later

Implementation: **`Queue`** with an **`Array`** - **`enqueue()`**

- If the queue is empty, **`first`** and **`last`** must be set to **`-1`**
- Otherwise, **`array`** is **full** exactly when **`element`** should be inserted at the position **`first`** (attention: we cannot assume **`first == 0`**)
- In this case we create a new array with double size
  We copy the elements

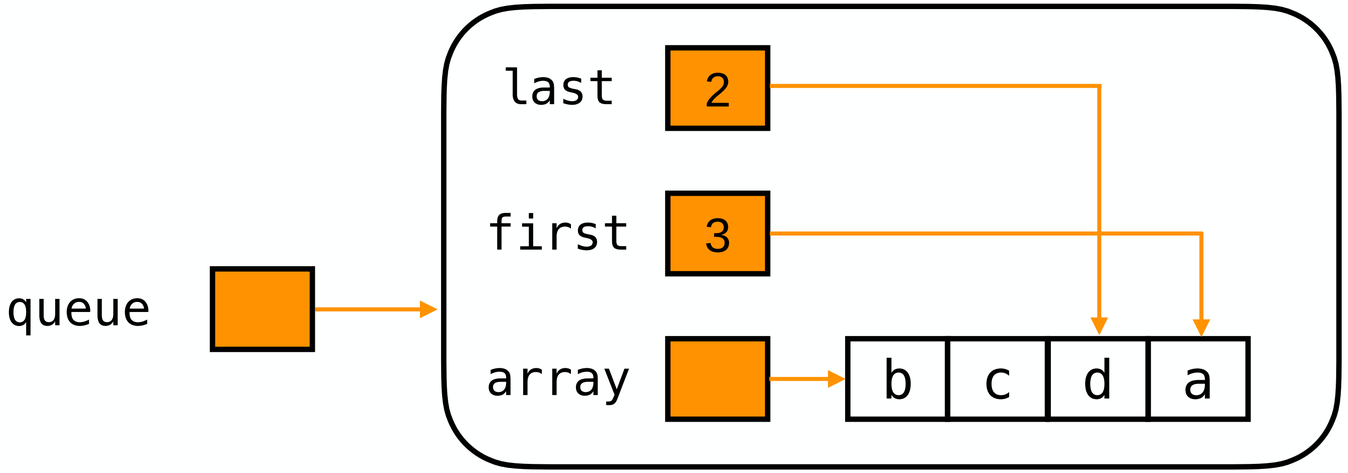
`first` and **`(last + 1) % length`** would have the same value

```
array[first], array[first + 1], ..., array[length - 1], array[0], array[1], ..., array[first - 1]
```

to `newArray[0],..., newArray[length - 1]`

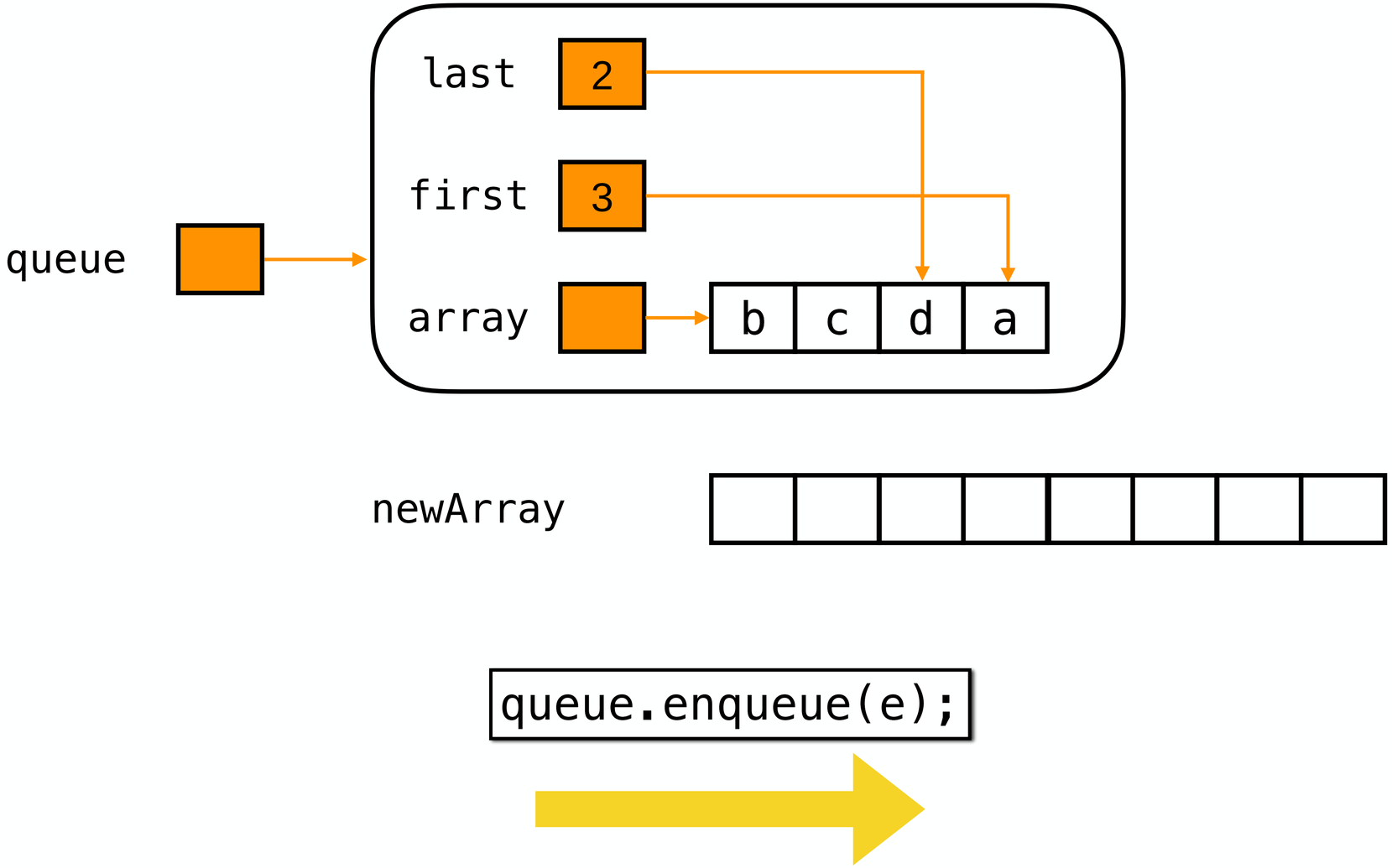
- Then `first = 0;` `last = array.length;` `array = newArray;`
- Now **`element`** can be stored at the position **`array[last]`**

Example process: doubling the **`Array`** size

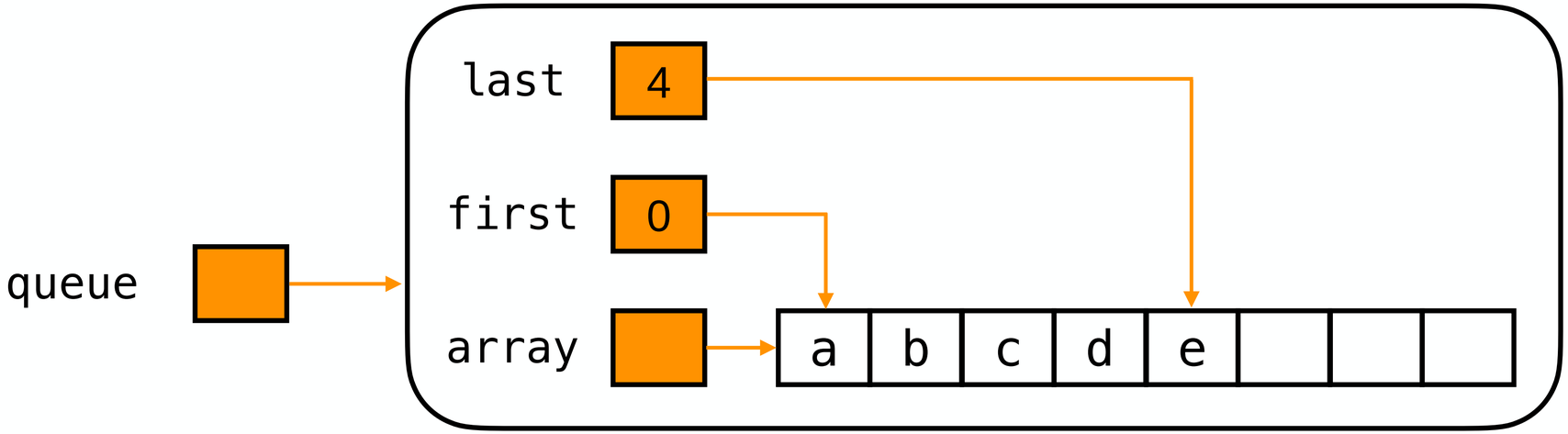



```
queue.enqueue(e);
```

Example process: doubling the **`Array`** size



Example process: doubling the **`Array`** size



Implementation: **Queue** with an **Array** - **enqueue()**

```
public void enqueue(Object element) {
    if (first == -1) {
        first = 0;
        last = 0;
    } else {
        int length = array.length;
        last = (last + 1) % length;
        if (last == first) {
            // queue full
            Object[] newArray = new Object[2 * length];
            for (int i = 0; i < length; i++) {
                newArray[i] = array[(first + i) % length];
            }
            first = 0;
            last = length;
            array = newArray;
        }
    }
    array[last] = element;
}
```

Implementation: **`Queue`** with an **`Array`** - **`dequeue()`**

- If after removing **`array[first]`** the queue is empty, **`first`** and **`last`** are set to **`-1`**
- Otherwise, **`first`** is incremented by 1 (modulo **`array.length`**)

```
public Object dequeue () {
    // assumption: first != -1
    Object result = array[first];
    if (first == last) {
        first = -1;
        last = -1;
    } else {
        first = (first + 1) % array.length;
    }
    return result;
}
```

Discussion

- In this implementation of **`dequeue()`**, the queue space is never reduced
- If the number of elements in the queue falls below a quarter of the length of **`array`**, we can replace it with another one with half the size (as with stacks)
- Attention: the elements in the queue do not need to be only at the beginning of **`array`**

Improved **`toString()`** method

```java
@Override
public String toString() {
    if (isEmpty()) {
        return "[]";
    }

    StringBuilder builder = new StringBuilder();
    builder.append("[");

    for (int i = first; i != last; i = (i + 1) % array.length) {
        builder.append(array[i]);
        builder.append(", ");
    }

    // Append the last element (since loop stops before it)
    builder.append(array[last]);
    builder.append("]");

    return builder.toString();
}
```

Efficient way to deal with **mutable** strings in loops

Start at **first** and append all elements until **last** potentially jumping over the end of the array to the beginning by using **`%`**

Exercise

10 min

- Copy the code into your Playground
- Instantiate a **`queue`** based on an **`array`**
- Enqueue and dequeue multiple elements (e.g. Strings)
- Debug how **`enqueue`** and **`dequeue`** work to better understand these operations

Note: you can copy the Queue code from https://gist.github.com/krusche/4d2d8be6d81516258813bdc968330381

Next steps

- **Tutor group** exercises
  - T04E01 - The Seal Train 6
  - T04E02 - Halloween Party FIFO
- **Homework** exercises
  - H04E01 - Phantom Phonebook
  - H04E02 - Spooky Track Voyager
- Read the following articles
  - https://www.digitalocean.com/community/tutorials/collections-in-java-tutorial
  - https://www.tpointtech.com/difference-between-array-and-arraylist

→ Due until **Wednesday, November 12, 13:00**

Summary

- The data type **`List`** is flexible and suited for rapid prototyping
- **`LinkedList`** vs. **`ArrayList`** vs. **`Array`**
- There are multiple implementations for useful data types **`Stack (LIFO)`** and **`Queue (FIFO)`**
- Often, there are additional operations for data types
- The **built-in Java collection types** (e.g. **`List`**, **`Set`**, **`Map`**) offer a great starting point and customizations for special use cases
  - Based on generic data types (type safety, no casting)
- The **enhanced for loop** allows to iterate through collection types (be careful with concurrent modification)

References

- https://www.digitalocean.com/community/tutorials/collections-in-java-tutorial
- https://www.educba.com/java-list-vs-array-list
- https://www.tpointtech.com/difference-between-array-and-arraylist
- https://www.baeldung.com/java-queue
- https://www.geeksforgeeks.org/stack-class-in-java